Отчет по лабораторной работе №3

Транспонирование матрицы и исследование локальности памяти

Выполнила: Шакирова Асиля Ришатовна, 11-401

Система и условия

Apple M1 Max, 10 ядер (8 производительных и 2 энергоэффективных), 64 ГБ памяти. Кеши: L1d 128 КБ (у энергоэффективных ядер 64 КБ), L2 12 МБ (у энергоэффективных 4 МБ), L3 в sysctl нет (по открытым данным, общий кеш системного уровня 48 МБ); кеш-линия 128 Б. Компилятор Apple clang 17.0.0 с libomp, ключ -O2. Вариант H: float, N_v = 5500. В таблицах медиана 5 запусков после одного прогревочного; rows_omp запускался со schedule(static).

Эксперимент 1. Порядок обхода (1 поток)

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0,000137	0,000092	3,82	5,70
512	0,001231	0,000256	1,70	8,19
1024	0,004065	0,000971	2,06	8,64
2048	0,019462	0,013139	1,72	2,55
5500	0,109239	0,086981	2,22	2,78

Эксперимент 2. Размер блока (1 поток, N = 5500)

Размер блока b	Время, с	BW, ГБ/с
4	0,040401	5,99
8	0,030237	8,00
16	0,026937	8,98
32 (b*)	0,020960	11,55
64	0,022956	10,54
128	0,021666	11,17

Эксперимент 3. Потоки (N = 5500, b = 32)

Потоки	Время rows_omp, с	S rows_omp	Время blocked_omp, с	S blocked_omp
1	0,110210	1,00	0,020596	5,34
2	0,057688	1,91	0,011107	9,90
4	0,031782	3,46	0,007778	14,14
8	0,016233	6,78	0,006335	17,36
16	0,016427	6,70	0,006251	17,60

$S = T_1 / T$, где T_1 = 0,109993 с (transpose_rows, 1 поток).

Эксперимент 4. False sharing (8 потоков, N = 5500)

Стратегия	Время, с	BW, ГБ/с
schedule(static)	0,016332	14,82
schedule(static, 1)	0,018181	13,31
schedule(static, 8)	0,017878	13,54

Дополнительно. На месте (N = 5500, b = 32)

Потоки	blocked_omp, с	inplace_omp, с
1	0,020679	0,013955
2	0,011329	0,008617
4	0,006990	0,006566
8	0,006388	0,003416
16	0,006196	0,003471

Вывод

1. В обеих версиях одна из матриц обходится с большим шагом (в rows запись в B, в cols чтение из A): каждое такое обращение попадает на новую кеш-линию, которая успевает вытесниться до использования соседнего элемента, а предвыборка плохо помогает, поэтому обе версии медленные (1,7-2,8 ГБ/с при N = 2048 и 5500). transpose_cols при этом быстрее при всех N, вероятно, потому что процессору легче скрыть задержки чтения с шагом, чем записи с шагом (отдельно это не проверялось). Разница заметна с N = 512 (в 4,8 раза), а при N = 2048 и 5500, когда матрицы уже не помещаются в L2, уменьшается до 1,5 и 1,3 раза.

2. b* = 32 (0,0210 с): пара блоков (из A и из B) должна помещаться в L1, то есть $2 \cdot b^2 \cdot 4$ Б ≤ 128 КБ, откуда b ≤ 128, и при b = 32, 64 и 128 время действительно почти одинаковое (0,0210, 0,0230 и 0,0217 с). При b = 4, 8 и 16 время хуже (0,0404, 0,0302 и 0,0269 с), потому что строка блока (16-64 Б) использует кеш-линию в 128 Б лишь частично, а при b = 32 строка блока равна одной кеш-линии. Разница между b = 32, 64 и 128 не больше 10%, поэтому b* = 32 лучший лишь в пределах разброса измерений.

3. Транспонирование ничего не вычисляет, а только перемещает данные (каждый элемент один раз читается и один раз записывается), поэтому потоки упираются в кеши и память, а в интеграле на каждое число приходится много вычислений (cos, sin). blocked_omp на 8 потоках ускоряется относительно своего одного потока лишь в 3,3 раза (rows_omp в 6,8 раза, интеграл в ЛР2 в 6,1).

4. С schedule(static, 1) время 0,0182 с против 0,0163 с со static (на 11% больше): при чередовании строк соседние потоки пишут в одну кеш-линию матрицы B, и она переходит между ядрами (false sharing), а при static каждый поток получает около 690 подряд идущих строк.